ПОЛНЫЙ ГАЙД · ОТ НУЛЯ ДО BRAIN

Как работать с ИИ

Подробная инструкция человеческим языком: что такое ИИ, промпт, контекст, проект, агент и Brain — с аналогиями «для пятилетнего» и глубиной для практики.

> **ГЛАВНАЯ МЫСЛЬ**
>
> ИИ предсказывает полезный текст. Ты задаёшь цель. Контекст даёт материалы. Агент даёт руки. Память сохраняет опыт. Проверка защищает от уверенной чепухи.

Содержание

1. Что такое ИИ
2. Как ИИ «думает»
3. Что решает ИИ
4. Что такое промпт
5. Как правильно писать промпты
6. Пространство / контекст
7. Проект и правила
8. Что такое агент
9. Brain и память
10. Продвинутый уровень
11. Шпаргалка

01 · Что такое ИИ

> **КАК 5-ЛЕТНЕМУ**
>
> ИИ — это программа, которую научили на куче текстов, картинок и примеров. Как ребёнок, который прочитал миллион книжек и теперь умеет отвечать на вопросы, сочинять истории и помогать с делами. Он не «живёт» и не «чувствует», но очень хорошо имитирует полезный разговор.

Три слоя, которые путают все

Слово	Простыми словами	Пример
ИИ (AI)	Любая система, которая делает «умные» вещи	Рекомендации видео, перевод, чат-бот
ML / обучение	Компьютер сам находит закономерности в данных	Модель учится отличать кошку от собаки
LLM / большая языковая модель	ИИ, который предсказывает следующий кусочек текста	ChatGPT, Claude, Gemini, модели в Cursor

> **ГЛУБЖЕ**
>
> Современные чат-ИИ — это в основном LLM: нейросети-трансформеры, обученные предсказывать следующий токен (кусочек слова) по всему предыдущему тексту. Из этого «угадывания по очереди» получается связный ответ, код, план, письмо. Есть и другие виды ИИ: зрение, речь, рекомендации. В быту «работа с ИИ» обычно значит чат с языковой моделью плюс инструменты вокруг неё.

Аналогия кухни

Модель

Шеф-повар с огромным опытом рецептов в голове.

Промпт

Твой заказ: что приготовить, для кого, какой вкус.

Контекст

Продукты на столе: файлы, правила, история чата.

02 · Как ИИ «думает»

> **КАК 5-ЛЕТНЕМУ**
>
> Представь автодополнение в телефоне, только супер-мощное. Ты пишешь начало — ИИ подбирает самые вероятные продолжения слово за словом, пока не получится целый ответ.

Шаг за шагом

1. Твой текст режут на токены (кусочки слов).
2. Модель смотрит на весь доступный контекст.
3. Считает вероятности следующего токена.
4. Выбирает один (иногда с долей случайности — temperature).
5. Повторяет, пока не закончит ответ.

> **ГЛУБЖЕ**
>
> У модели нет отдельной «базы фактов как Википедия внутри головы». Знания сжаты в веса нейросети. Поэтому она может уверенно сказать неправду — это галлюцинация: текст звучит правдоподобно, но факт выдуман. Reasoning-модели сначала рассуждают во внутреннем черновике, потом отвечают — это помогает в логике, но не гарантирует истину.

Почему один и тот же вопрос даёт разные ответы

Причина	Что это значит для тебя
Случайность (temperature)	Можно попросить «будь строже / креативнее»
Разный контекст	Вчерашние файлы и правила меняют ответ
Другая модель	У моделей разный стиль и сильные стороны
Сжатие старого чата	Длинный диалог «забывает» ранние детали

> **ВАЖНО**
>
> Не путай уверенность с правдой. ИИ почти всегда пишет спокойно и уверенно — даже когда ошибается. Для важных фактов: проси источники, проверяй числа, сверяй код тестами.

03 · Что решает ИИ

> **КАК 5-ЛЕТНЕМУ**
>
> ИИ хорошо помогает придумать, объяснить, набросать и поправить. Но важные решения — «покупать или нет», «увольнять или нет», «это правда?» — остаются за взрослым человеком. ИИ — советник, не начальник.

ИИ отлично решает ДА

- Черновики текстов, писем, планов
- Объяснение сложного простыми словами
- Написание и правка кода по образцу
- Разбор большого текста на тезисы
- Варианты идей и структуры
- Пошаговые инструкции
- Поиск багов при наличии логов и файлов

Плохо / нельзя отдавать НЕТ

- Финальную юридическую / медицинскую ответственность
- Секреты, пароли, ключи API в открытый чат
- «Единственную правду» без проверки источников
- Действия в реальном мире без твоего контроля
- Задачи без доступа к нужным свежим данным
- Моральный выбор за тебя

Кто принимает решение в связке человек + ИИ

Слой	Кто решает	Пример
Цель	Ты	«Нужен лендинг для курса»
Способ / план	ИИ предлагает, ты утверждаешь	Структура страницы
Детали исполнения	ИИ	Текст, код, правки
Проверка	Ты + тесты / факты	Запуск, ревью, сверка
Публикация / деньги / люди	Только ты	Деплой, оплата, письмо клиенту

> **ГЛУБЖЕ**
>
> Правильная установка: ИИ ускоряет черновую работу и расширяет варианты. Ты задаёшь цель, ограничения и критерии качества. Чем яснее критерии («успех = …»), тем меньше ИИ «решает за тебя» наугад.

04 · Что такое промпт

> **КАК 5-ЛЕТНЕМУ**
>
> Промпт — это твоя просьба к ИИ. Как записка другу: чем яснее напишешь, тем точнее он поможет. «Сделай что-нибудь красивое» — плохая записка. «Нарисуй синий дом с двумя окнами для открытки бабушке» — хорошая.

Из чего состоит разговор с моделью

Часть	Кто пишет	Зачем
System / правила	Продукт + твои Rules	Постоянный характер и запреты
User prompt	Ты	Текущая задача
Контекст (@файлы, история)	Ты / агент	Материалы для ответа
Ответ модели	ИИ	Результат
Tool results	Инструменты	Факты из кода, веба, терминала

> **ГЛУБЖЕ**
>
> Промпт-инжиниринг — это умение упаковать задачу так, чтобы модель потратила внимание на нужное. Не «магия слов», а ясность: роль, цель, входные данные, формат выхода, ограничения, примеры. Для агентов хороший промпт ещё управляет поведением: что сделать сначала, когда спросить, что считать готовым.

> **МИФ**
>
> Не нужно разговаривать с ИИ «как с роботом» или наоборот «как с джинном». Нужно говорить как с очень способным стажёром: конкретно, с примерами и критериями.

05 · Как правильно писать промпты

> **КАК 5-ЛЕТНЕМУ**
>
> Хорошая просьба отвечает на вопросы: КТО ты хочешь чтобы помог? ЧТО сделать? ДЛЯ КОГО? КАКИМ должен быть ответ? ЧЕГО НЕ делать?

Формула R-T-C-F-O

Буква	Смысл	Подсказка
R	Role — роль	«Ты старший редактор / учитель / архитектор»
T	Task — задача	Один главный глагол: напиши, найди, исправь
C	Context — контекст	Факты, аудитория, файлы, ограничения
F	Format — формат	Таблица, список, код, 5 пунктов, JSON
O	Outcome — критерий успеха	«Готово, когда…» / «нельзя…»

Примеры

Слабо

«Напиши пост про ИИ»

Сильно

«Ты редактор Telegram-канала для новичков. Напиши пост на 800–1000 знаков: 3 ошибки новичков в промптах. Тон дружелюбный, без жаргона. В конце — один чек-лист из 5 пунктов.»

Слабо

«Почини баг»

Сильно

«В @auth.ts логин падает с 401 на staging. Разбери причину по коду и логам. Сначала гипотезы (макс. 3), потом минимальный фикс, потом как проверить. Код в стиле проекта, без рефакторинга рядом.»

Золотые приёмы

1. **Один главный результат на сообщение.** Не смешивай всё без приоритета.
2. **Покажи образец.** «В таком же стиле, как этот файл» лучше абстрактных слов.
3. **Заставь проверить себя.** «Сначала риски, потом решение. Отметь, где не уверен.»
4. **Режь объём.** «Максимум 10 строк» или «только diff» повышает точность.
5. **Итерируй.** Первый ответ = черновик. Второй: «короче», «жёстче», «добавь пример».

Шаблон, который можно копировать

Роль: …

Задача: …

Контекст: … (файлы / факты / аудитория)

Ограничения: не делай …; используй …

Формат ответа: …

Критерий готовности: …

---

06 · Пространство / контекст

> **КАК 5-ЛЕТНЕМУ**
>
> У ИИ есть «стол», на который помещается только ограниченная куча листочков: твоя просьба, прошлые сообщения, файлы, правила. Это пространство контекста (context window). Если листочков слишком много — старые убирают или сжимают в краткий пересказ.

Что лежит на «столе»

Кусок	Обычно содержит
System / product prompt	Как модели вести себя в этом продукте
Rules / instructions	Твои постоянные правила
Skills / tools / MCP	Описание доступных инструментов
История чата	Сообщения туда-сюда
Вложения и @упоминания	Файлы, диффы, терминал
Ответы инструментов	Результаты поиска, чтения файлов, команд

ГЛУБЖЕ

В Cursor рядом с полем ввода есть кольцо контекста: видно, сколько места занято system prompt, tools, rules, skills, MCP, саммари старого чата и текущим разговором. Когда окно почти полное, старые части сжимаются. «Пространство» также называют workspace — папку проекта на диске. Не путай: context window = память текущего разговора; workspace = место, где лежат файлы работы.

КАК НЕ ЗАБИТЬ СТОЛ

- Клади только нужные файлы, не весь репозиторий сразу.
- Длинные чаты начинай заново для новой задачи.
- Важные решения выноси в Rules / заметки / Brain — не держи только в чате.
- Пиши коротко: меньше воды в промпте = больше места на дело.

---

07 · Проект и правила

КАК 5-ЛЕТНЕМУ

Проект — это коробка с игрушками на одну большую игру: все связанные чаты, файлы и правила лежат вместе. Тогда ИИ не путает «домашку по математике» с «рисунком для открытки».

Без проекта

Каждый чат начинается почти с нуля. Ты снова объясняешь кто ты, какой тон, какие файлы важны.

С проектом

Общие инструкции, документы и история темы уже «в коробке». Новые чаты внутри проекта стартуют умнее.

Три разных «инструкции», которые путают

Механизм	Что хранит	Когда использовать
Custom / User instructions	Как отвечать тебе всегда	Стиль: кратко, на русском, без воды
Project instructions	Правила только этой работы	Термины продукта, аудитория, форматы
Memory / Brain	Факты о тебе и решениях	«Я в Москве», «мы выбрали Postgres»

> **ГЛУБЖЕ**
>
> В ChatGPT Projects — рабочие пространства с чатами, файлами и project instructions; инструкции проекта перекрывают глобальные custom instructions внутри проекта. В Cursor «проектом» обычно является репозиторий. Постоянные указания кладут в User Rules, Project Rules (`.cursor/rules`) и иногда `AGENTS.md` . Правила попадают в начало контекста и задают стабильное поведение агента.

Как писать хорошие правила

- Коротко и проверяемо: «используй TypeScript», не «пиши красиво».
- Давай путь к каноническому примеру вместо копипасты гайда.
- Пиши запреты явно: «не трогай /billing без спроса».
- Добавляй правило только когда ИИ ошибается повторно.

---

08 · Что такое агент

> **КАК 5-ЛЕТНЕМУ**
>
> Обычный чат — это умный советчик за столом: спросил — ответил. Агент — это помощник, которому можно сказать «убери комнату»: он сам ищет пылесос, включает его, проверяет угол и докладывает, что готово. Он действует шагами и пользуется инструментами.

Чат vs агент

	Чат	**Агент**
Главное действие	Генерирует текст	Планирует + вызывает инструменты
Доступ к миру	Только то, что в сообщении	Файлы, терминал, браузер, MCP
Цикл	1 вопрос → 1 ответ	Думает → действует → смотрит результат → снова
Риск	Неверные советы	Неверные действия (нужен контроль)
Когда выбирать	Идея, черновик, объяснение	Сделать задачу до результата

Типичный цикл агента

1. Понять цель и ограничения.
2. Найти нужный контекст (поиск по коду, файлы, доки).
3. Сделать шаг (правка, команда, запрос).

4. Прочитать результат / ошибку.
5. Повторять, пока критерий готовности не выполнен.
6. Кратко отчитаться человеку.

> **ГЛУБЖЕ**
>
> В Cursor Agent работает в чате, но может читать и менять файлы, запускать команды, ходить в браузер, звать сабагентов. Ты направляешь его промптом и @упоминаниями файлов; если не знаешь какой файл нужен — часто лучше не угадывать: агент умеет искать сам. Агентность = автономия + инструменты + обратная связь. Без проверки агент легко уверенно ломает вещи.

> **КАК СТАВИТЬ ЗАДАЧУ АГЕНТУ**
>
> - Цель: что должно получиться в конце.
> - Границы: какие файлы/системы нельзя трогать.
> - Проверка: как понять, что готово (тест, скрин, команда).
> - Стиль работы: «сначала план», «сразу патч», «спроси если риск».

---

09 · Brain и память

> **КАК 5-ЛЕТНЕМУ**
>
> Обычный ИИ как рыбка: вышел из комнаты — забыл игру. Brain / Memory — это блокнот снаружи: важные факты записывают и потом находят снова, даже в новом разговоре.

Языковая модель сама по себе не хранит твои прошлые сессии между чатами. Помнит только то, что сейчас лежит в контексте, плюс то, что продукт специально сохранил: memories, rules, project files, внешний Brain.

Слои памяти

Слой	Живёт где	Хорошо для
Контекст чата	Текущее окно	Детали этой задачи
Summary	Сжатие старого чата	Продолжить длинный диалог
Product Memory	Профиль в ChatGPT и др.	Предпочтения о тебе
Rules / AGENTS.md	Репозиторий / настройки	Как работать всегда
Project files	Коробка проекта	Документы конкретной работы
Brain / MCP memory	Внешний граф/база	Факты и решения между инструментами

> **ГЛУБЖЕ**
>
> Brain (например INITE Brain или MCP memory-серверы) — внешняя долговременная память: факты, эпизоды, источники. Клиенты (Cursor, ChatGPT, Claude) подключаются к одному графу. Что записал один — может вспомнить другой. Brain не заменяет агента: он хранит и достаёт контекст. Rules говорят КАК работать; память хранит ЧТО решили и узнали.

Клади в память ДА

- Архитектурные решения и почему
- Стабильные предпочтения
- Термины и «как принято у нас»
- Грабли, на которые уже наступили

Не клади в память НЕТ

- Секреты, пароли, ключи
- Огромные куски кода
- Одноразовые to-do на сегодня
- Непроверенные догадки как факты

> **ПРАКТИЧЕСКОЕ ПРАВИЛО**
>
> Если факт должен пережить закрытие чата — сохрани его явно: в Rules, README, проектный файл или Brain. Не надейся, что «он и так помнит».

10 · Продвинутый уровень

> **КАК 5-ЛЕТНЕМУ**
>
> Когда помощник научился слушать, ему дают инструменты: лупу (поиск), руки (терминал), записную книжку (память), телефон до других служб (MCP). Тогда он не только болтает — он работает.

Ключевые слова взрослых

Термин	Простыми словами
Tools / function calling	ИИ может вызвать функцию: поиск, файл, API
RAG	Сначала найти куски документов, потом отвечать по ним
MCP	Стандарт «розеток» для внешних сервисов у агента
Skills	Готовые рецепты для типовых задач агента
Subagents	Помощники на подзадачи
Eval / hillclimb	Измерять качество и улучшать системно
Temperature / top-p	Настройки случайности генерации
Grounding	Привязка ответа к источникам, а не к догадкам

Сильный рабочий контур

1. Цель и критерий успеха (что значит «готово»).
2. Контекст: проект + нужные файлы + правила.
3. Агент с инструментами делает работу.
4. Проверка: тесты, факты, ревью.
5. Память: решение записали в Rule/Brain/доки.
6. Новый чат для новой большой задачи — чистый стол.

> **ГЛУБЖЕ**
>
> Самый большой рывок качества обычно даёт не «хитрый промпт», а система: хорошие правила, узкий контекст, инструменты с фактами, проверка результата. Промпт остаётся рулём, но машина должна быть собранной.

Мини-ритуал перед сложной задачей

1. Зачем это нужно человеку? (цель)
2. Что уже известно / где файлы?
3. Что нельзя ломать?
4. Как проверим успех за 2 минуты?
5. Нужно ли сохранить вывод в память проекта?

---

11 · Шпаргалка

Чек-лист сильного запроса

- Есть одна главная задача
- Понятна аудитория / роль
- Дан нужный контекст (не весь мир)
- Задан формат ответа
- Есть ограничения и критерий готовности
- Для агента: способ проверки

Типичные ошибки

Ошибка	Что делать вместо
«Сделай хорошо»	Дать пример и критерий
Огромный портянка-промпт	Коротко + ссылки на файлы
Один вечный чат на всё	Новый чат на новую цель
Верить уверенному тону	Проверять факты и прогонять тесты
Пихать секреты в чат	Использовать env / секретницу
Правила на 2000 строк	Короткие правила + ссылки на примеры
Ждать, что ИИ «сам помнит»	Rules / Project / Brain

Карманный словарь

Промпт

Твоя инструкция в этом ходе.

Контекст / пространство

Всё, что модель видит прямо сейчас.

Проект

Коробка чатов, файлов и правил одной работы.

Агент

ИИ, который действует инструментами по циклу.

Rules

Постоянные указания «как работать».

Brain

Внешний блокнот фактов между сессиями.

итог

ИИ предсказывает полезный текст. Ты задаёшь цель. Контекст даёт материалы. Агент даёт руки. Память сохраняет опыт. Проверка защищает от уверенной чепухи. Освой эти пять рычагов — и ты уже на взрослом уровне.

Гайд подготовлен как учебный материал. Продуктовые детали Cursor / ChatGPT / Brain могут меняться — сверяй с актуальной документацией инструментов.